

Personality Type Prediction

The aim of this project is to accurately predict whether respondents identify themselves as introverts or as extroverts based on self-reported data relating to respondents' social habits, such as frequency of social media posts, friend group sizes, and time spent alone.

About the dataset

The dataset was published by Kaggle in 2025 and comprises 9 variables and 18,524 observations

Column Name	Description
id	Unique identifier assigned to each respondent
Time_spent_Alone	Likert scale measure (0 to 10) of how much time the respondent spends alone
Stage_fear	Likert scale measure (0 to 10) of how much stage fright the respondent experiences
Social_event_attendance	Likert scale measure (0 to 10) of how frequently the respondent attends social events
Going_outside	Likert scale measure (0 to 10) of how much time the respondent spends in outdoor spaces
Drained_after_socializing	Binary measure (yes/no) of whether socialising fatigues the respondent
Friends_circle_size	Continuous measure of how many close friends the respondent has
Post_frequency	Likert scale measure (0 to 10) of how frequently the respondent makes social media posts
Personality	Binary measure (introvert/extrovert) of respondent's personality type

Exploratory data analysis (EDA)

Through an EDA of the dataset, I want to discover if there are:

- Any missing values?
 - If so, are the values missing completely at random (MCAR), missing at random (MAR), or missing not at random (MNAR)?
- Any clear correlations between features?

Answering these questions will help me to decide on a strategy for both cleaning and modelling the data.

Missing values

```
#Load libraries
library(tidyverse) #For data cleaning and manipulation
library(stargazer) #For regression tables
library(naniar) #To visualise missing values
library(mice) #For imputation
library(pscl) #For pseudo R2

set.seed(1234) #For reproducibility

#Load the dataset
IntroExtro <- read.csv("D:\\Datasets\\IntrovertExtrovertDatasetTrain.csv")
tibble(IntroExtro)
```

Out:

```
# A tibble: 18,524 × 9
   id Time_spent_Alone Stage_fear Social_event_attendance Going_outside
  <int>          <dbl> <chr>                <dbl>          <dbl>
1     0             0 No                     6             4
2     1             1 No                     7             3
3     2             6 Yes                    1             0
4     3             3 No                     7             3
5     4             1 No                     4             4
6     5             2 No                     8             5
7     6             1 No                     8            NA
8     7             2 No                     8             3
9     8             4 Yes                    2             1
10    9             1 No                     8             6
# i 18,514 more rows
# i 4 more variables: Drained_after_socializing <chr>, Friends_circle_size
<dbl>,
```

```
# Post_frequency <dbl>, Personality <chr>
```

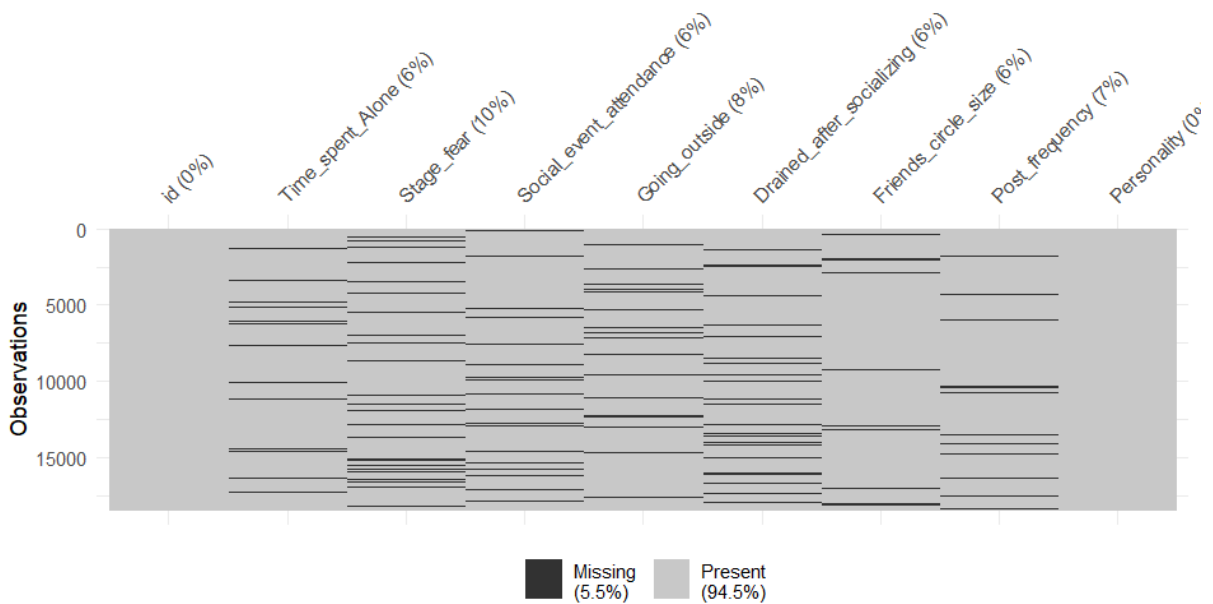
The dataset has loaded in correctly, and, from the `tibble()` call, it also looks as though missing values are present in this dataset.

```
#Check binary variables for blank values
unique(IntroExtro$Stage_fear)
unique(IntroExtro$Drained_after_socializing)

#Convert blank values to NAs
IntroExtro$Stage_fear <- case_when(IntroExtro$Stage_fear == "" ~ NA,
                                   TRUE ~ IntroExtro$Stage_fear)
IntroExtro$Drained_after_socializing <-
case_when(IntroExtro$Drained_after_socializing == "" ~ NA,
          TRUE ~ IntroExtro$Drained_after_socializing)

#Find NAs
vis_miss(IntroExtro)
colSums(is.na(IntroExtro))
```

Out:



	id	Time_spent_Alone	Stage_fear
	0	1190	1893
Social_event_attendance		Going_outside	Drained_after_socializing
	1180	1466	1149
Friends_circle_size		Post_frequency	Personality
	1054	1264	0

The `colsums()` and `vis_miss()` calls confirm that missing values are indeed prevalent in the data, amounting to 5.5% of the entire dataset. While this is relatively minor, simply removing the observations that contain NA values is nonetheless likely to weaken the predictive power of our model.

An alternative strategy would be to impute the missing values. There are numerous imputation methods – some of which rely on simple mean calculations, while others utilise more complex regression techniques to predict the most likely value, had it not been missing.

The strategy for handling missing data should depend on how it is *structured*. If the data is missing totally at random, for instance, it may make more sense to either remove the observations that contain missing values entirely, or use a simple mean imputation method. If the missingness is *patterned*, however, it may be more effective to impute missing values with a predictive model, thereby ensuring greater precision and preserving external validity.

```
#Convert to binary variables factors for glm() and mice
IntroExtro$Stage_fear <- as.factor(IntroExtro$Stage_fear)
IntroExtro$Drained_after_socializing <-
as.factor(IntroExtro$Drained_after_socializing)

#Check if missingness is random or patterned
models <- list() #Create list to store regression models

for (i in colnames(IntroExtro)) { #Loop through columns in the data
  if (sum(is.na(IntroExtro[[i]])) > 0) { #Check if there are NAs in the given
column
    models[[i]] <- glm(formula = as.formula(paste('is.na(', i, ') ~ .')), family
= "binomial", data = IntroExtro) #If NAs are present, build a regression model
to determine if the NAs are predictable
  }
}
```

```
for (i in models) { #Loop through models
  print(summary(i)) #Print summaries to check p-values
}
```

A visual inspection of the output from the `summary()` call confirms that the missing values are indeed significantly related to the rest of the data for every variable tested. For instance, nearly all variables in the dataset are significantly correlated with the missing value incidence in 'Stage_fear':

```
Call:
glm(formula = as.formula(paste("is.na(", i, ") ~ .")), family = "binomial",
    data = IntroExtro)

Coefficients:
                Estimate Std. Error z value Pr(>|z|)
(Intercept)    -1.140e+00  1.914e-01  -5.955 2.61e-09 ***
id              4.914e-06  5.108e-06   0.962 0.336124
Time_spent_Alone -1.941e-02  1.527e-02  -1.272 0.203529
Social_event_attendance -5.100e-03  1.542e-02  -0.331 0.740881
Going_outside   -1.662e-01  2.091e-02  -7.949 1.88e-15 ***
Drained_after_socializingYes -1.372e-02  2.008e-01  -0.068 0.945533
Friends_circle_size 4.105e-02  9.275e-03   4.426 9.61e-06 ***
Post_frequency   -9.335e-02  1.444e-02  -6.464 1.02e-10 ***
PersonalityIntrovert 5.044e-01  1.376e-01   3.664 0.000248 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 9482.6 on 11818 degrees of freedom
Residual deviance: 9006.1 on 11810 degrees of freedom
(6705 observations deleted due to missingness)
AIC: 9024.1

Number of Fisher Scoring iterations: 5
```

This finding suggests that the missing data is **not** missing completely at random (MCAR), indicating that it may instead be missing at random (MAR), meaning that the missingness is systematically related to other variables in the dataset. There is no evidence, however, to suggest that the missingness is related to the missing values themselves.

One drawback to recognise here is that the data has been imputed *before* splitting up the train and test data, which constitutes a form of leakage. However, given that the incidence of missing values is relatively small, this is unlikely to have affected the model in any drastic terms.

```
#Impute missing values
ImputedIntroExtro <- mice(IntroExtro, m = 5) #Use multiple imputation by chained
equations (MICE) regression to impute the data

CompletedIntroExtro <- complete(ImputedIntroExtro, 1) #Convert the MICE object
into a data frame
```

Feature distribution

```
#Inspect feature distribution
plotter <- function(i){ #Define a function that build plots for each variable
  if (is.numeric(i) == TRUE){ #Check if the variable is numeric
    ggplot(data = CompletedIntroExtro, mapping = aes(x = .data[[i]])) +
    geom_histogram(aes(fill = Personality)) #If it is numeric, build a histogram
  } else if (is.character(i) == TRUE){ #Check if the variable is a character
    ggplot(data = CompletedIntroExtro, mapping = aes(x = .data[[i]])) +
    geom_bar(aes(fill = Personality)) #If it is a character, build a bar plot
  }
}

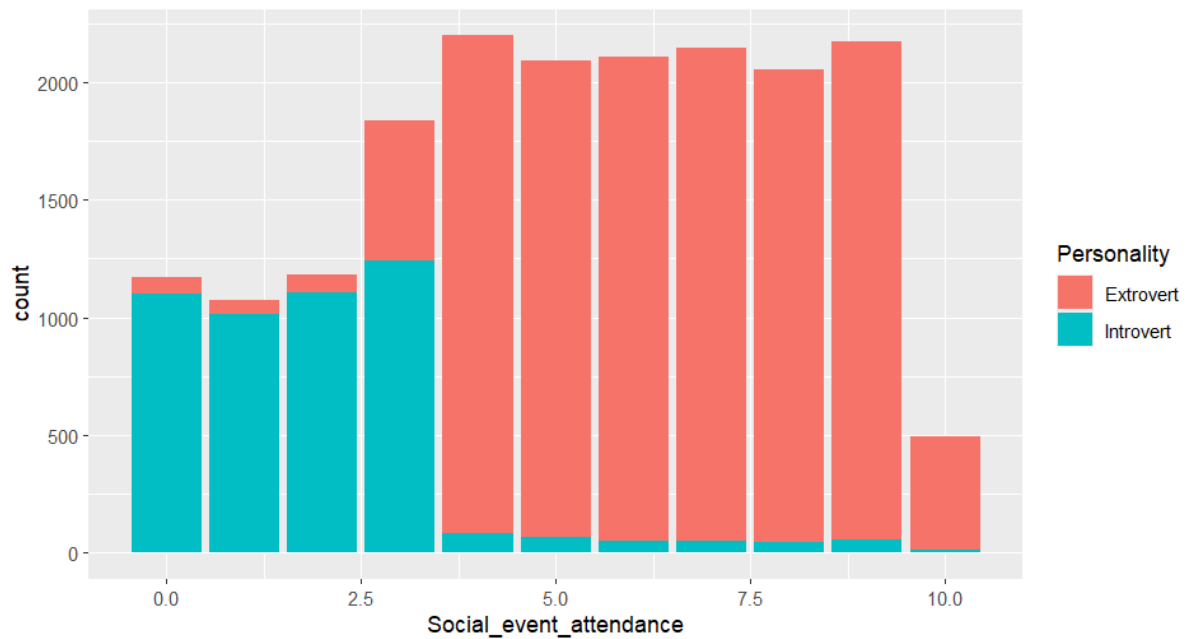
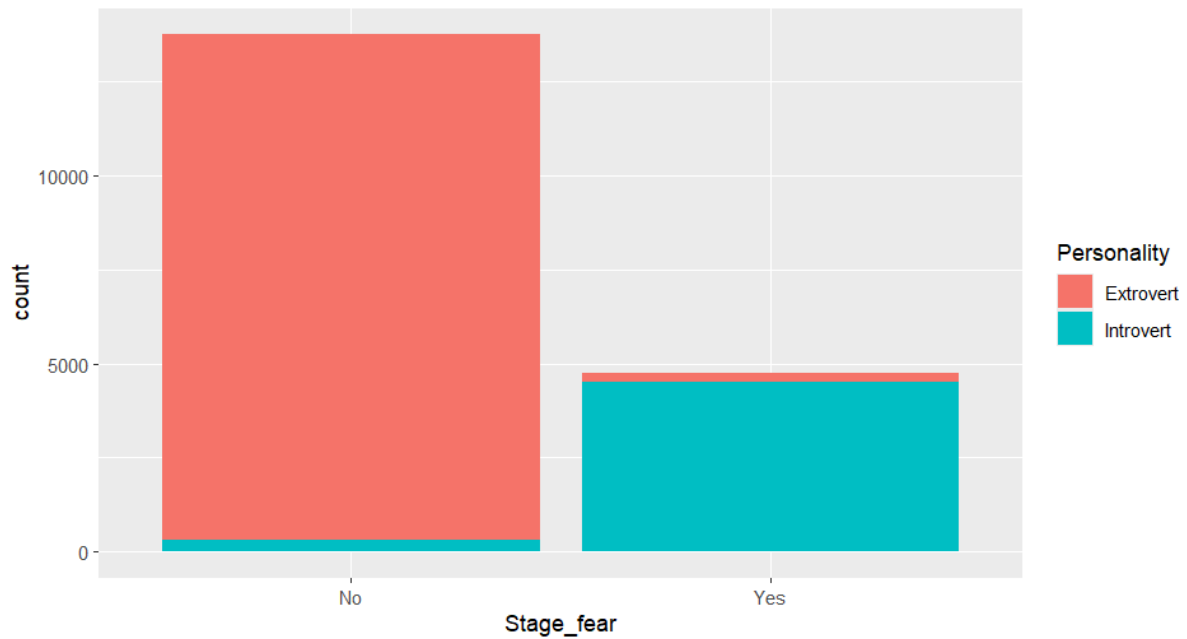
plots <- list() #Create a list to store the plots that plotter() builds

for (i in colnames(CompletedIntroExtro)) { #Loop through the columns
  plots[[i]] <- plotter(i) #Run plotter() on the given variable and save the
  plot to the list so it can be retrieved
}
```

A visual inspection of the plots reveal a very clear correlation between personality and the other variables in the dataset. For instance:

```
#Inspect some plots
plots$Social_event_attendance
plots$Stage_fear
```

Out:



The Stage_fear plot shows a huge bias in favour of introverts for stage fright.

Similarly, the Social_event_attendance plot shows a very clear correlation between personality and attending social events, as expected. With introverts attending far fewer social events than extroverts.

These feature distributions suggest that a linear model like logistic regression would be suitable for this project.

Modelling the data

Logistic regression is highly interpretable, works well on data with clustered feature distributions, and tends to resist overfitting on small datasets. Logistic regression is, therefore, likely to yield strong results here.

```
#Impute missing values
ImputedTest <- mice(TestIntroExtro, m = 5)
CompletedKaggleTest <- complete(ImputedTest, 1)
```

```
#Split training and testing data
CompletedIntroExtro$Personality <- case_when(CompletedIntroExtro$Personality ==
"Introvert" ~ 0,
                                             CompletedIntroExtro$Personality == "Extrovert" ~
1) #Convert target variable to numeric so LR can parse it
Train <- sample_frac(CompletedIntroExtro, 0.7) #Dedicate 70% of the data for
training and 30% for testing
Test <- anti_join(CompletedIntroExtro, Train, by = "id")

#Fit the model
BaselineGLM <- glm(formula = Personality ~ . - id, family = "binomial", data =
Train)

#Produce a regression table with Pseudo R^2 to assess fit
stargazer(
  BaselineGLM,
  type = "html",
  out = "BaselineGLM.html",
  add.lines = list(c(
    "McFadden pseudo R^2",
    round(pR2(BaselineGLM)["McFadden"], 3))
  ))
```

The regression table for the model indicates strong correlations between the predictors and the dependent variable, indicating that it likely possesses good predictive power. Moreover, pseudo R^2 is calculated as 0.771, suggesting that the model can explain around 77% of variation in the data. This is a great result and indicates that the model may be ready for predictive work.

	<i>Dependent variable:</i>
	Personality
Time_spent_Alone	-0.097*** (0.027)
Stage_fearYes	-3.114*** (0.391)
Social_event_attendance	0.055* (0.031)
Going_outside	-0.111*** (0.041)
Drained_after_socializingYes	-3.022*** (0.396)
Friends_circle_size	0.009 (0.019)
Post_frequency	0.034 (0.029)
Constant	3.852*** (0.363)
McFadden pseudo R ²	0.771
Observations	12,967
Log Likelihood	-1,702.014
Akaike Inf. Crit.	3,420.028
<i>Note:</i>	* p<0.1; ** p<0.05; *** p<0.01

Now that we can be confident the model was fit successfully, we can validate its performance with the test data.

```
#Validate the model
TestProbs <- predict(BaselineGLM, newdata = Test, type = "response")
TestPreds <- ifelse(TestProbs > 0.5, 1, 0)

#Accuracy score
mean(TestPreds == Test$Personality)
```

Out:

```
[1] 0.9672485
```

This test indicates that the model was around 97% accurate on unseen data,

suggesting that it generalises very well.

Now let's predict personality on the unseen Kaggle data and get a final score.

```
#Predict personality
CompletedKaggleTest$PersonalityProbs <- predict(BaselineGLM,
CompletedKaggleTest, type = "response") #Assign probabilities
CompletedKaggleTest$Personality <- ifelse(CompletedKaggleTest$PersonalityProbs >
0.5, 1, 0) #Assign a threshold at 0.5, where below the threshold indicates an
introvert, and above it an extrovert

CompletedKaggleTest <- subset(CompletedKaggleTest, select = -c(PersonalityProbs,
Drained_after_socializing, Friends_circle_size, Stage_fear, Going_outside,
Time_spent_Alone, Post_frequency, Social_event_attendance)) #Remove excess
columns for Kaggle submission

#Rename binary labels
CompletedKaggleTest$Personality <- case_when(CompletedKaggleTest$Personality ==
0 ~ "Introvert",
                                           CompletedKaggleTest$Personality == 1 ~
"Extrovert")

#Save predictions
write.csv(CompletedKaggleTest, 'IntExtBaselineGLM.csv', row.names = FALSE)
```

Score: 0.974089

The model was around 97% accurate on Kaggle's testing data – a success!